

[Accedi](#)[Registrati gratuitamente](#)

entity, gameloop, gameworld, rendering, e altro. Ho una situazione in cui quando i giocatori raggiungono automaticamente una posizione, dopo 2 secondi deve essere sparata una palla. Va bene usare il set timeout o è meglio evitarlo?



In generale: `setTimeout` **va bene, ma in un gioco "strutturato" spesso è meglio evitarlo per la logica di gameplay.**

Dipende molto da quanto vuoi che il tuo gioco sia **deterministico e sincronizzato con il game loop.**

Quando `setTimeout` è OK

Può andare bene se:

- il gioco è semplice o single-player casual
- l'azione "dopo 2 secondi" è puramente visiva o non critica
- non ti interessa la precisione al frame o la simulazione perfettamente riproducibile

Esempio tipico:

TypeScript

```
setTimeout(() => {  
  shootBall();  
}, 2000);
```

Problema: questo avviene fuori dal game loop.

Problemi di `setTimeout` in un game loop

In un'architettura come la tua (entity / gameworld / gameloop), `setTimeout` può creare:

- **desincronizzazione**: il gioco continua a simulare, ma l'evento arriva "da fuori"

- **problemi di pause:** se metti pausa, `setTimeout` continua comunque a contare
- **non determinismo:** difficile replicare lo stato (replay, multiplayer, rollback)
- **debug più complicato**

Approccio migliore: timer nel game loop (consigliato)

La soluzione più pulita è usare un **timer gestito dal game loop**.

Esempio con accumulatore in entity o system:

TypeScript

```
class Player {  
  state: "idle" | "moving" | "waitingToShoot" = "idle";  
  shootDelay = 2;  
  timer = 0;  
}
```

Nel game loop:

TypeScript

```
function update(dt: number) {  
  for (const player of players) {  
    if (player.state === "waitingToShoot") {  
      player.timer += dt;  
  
      if (player.timer >= player.shootDelay) {  
        shootBall(player);  
        player.state = "idle";  
        player.timer = 0;  
      }  
    }  
  }  
}
```

Alternativa migliore ancora: Event Scheduler nel GameWorld

Se vuoi qualcosa di più pulito:

```
class GameWorld {
  scheduledEvents: { time: number, callback: () => void }[] = [];
  time = 0;

  schedule(delay: number, cb: () => void) {
    this.scheduledEvents.push({
      time: this.time + delay,
      callback: cb
    });
  }

  update(dt: number) {
    this.time += dt;

    for (const e of this.scheduledEvents) {
      if (this.time >= e.time) {
        e.callback();
      }
    }

    this.scheduledEvents = this.scheduledEvents.filter(e => this.time < e.time);
  }
}
```

Uso:

TypeScript

```
gameWorld.schedule(2, () => shootBall(player));
```

Regola pratica

- ❌ setTimeout → ok solo per UI o effetti non legati alla simulazione
- ✅ game loop timer → per gameplay, fisica, AI, eventi
- ✅ scheduler interno → migliore soluzione per giochi medi/grandi

In breve

Per il tuo caso (azione gameplay dopo 2 secondi):

Se vuoi, posso aiutarti a disegnare un'architettura "pulita" per il tuo game loop (tipo ECS o system-based scheduler) adatta al tuo progetto in TypeScript.

